

Chasse au trésor

Limite de temps: 8 s Limite de mémoire: 256 MiB

Vous recherchez un trésor perdu depuis longtemps sur un immense champ de cases $N \times N$ à l'aide d'une boussole magique. Il y a au total $K \leq 3$ coffres au trésor cachés dans diverses cases distinctes du champ. Votre but est simple : les trouver tous !

Lorsque vous posez la boussole sur l'une des cases, elle trouve les plus courts chemins vers un coffre au trésor en utilisant uniquement des déplacements vers la gauche, le haut, la droite et le bas (c'est-à-dire un coffre au trésor le plus proche selon la distance de Manhattan). Elle indique ensuite la direction du premier déplacement. Si plusieurs premiers déplacements valides mènent à un coffre au trésor le plus proche (ou à plusieurs d'entre eux), la boussole les retourne tous. Si la case contient un trésor, la boussole l'indique à la place.

Chaque fois que vous trouvez un coffre au trésor, vous en videz le contenu, mais vous ne pouvez pas retirer le coffre – il est trop lourd. Votre boussole ne sait pas si un coffre est plein ou vide. Elle indique le chemin vers un coffre le plus proche, qu'il soit vide ou plein.

Essayez de trouver l'emplacement de tous les coffres au trésor en un minimum de requêtes de la boussole.

Tâche

Ceci est une tâche interactive. Dans chaque cas de test (c'est-à-dire chaque exécution de votre programme), votre programme devra résoudre plusieurs chasses au trésor. Vous devez interagir avec le grader en utilisant une bibliothèque fournie par les organisateurs. Cette bibliothèque contient les déclarations suivantes :

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — appelez cette fonction pour démarrer la prochaine chasse au trésor. Elle place la taille de la grille dans la variable *N* et le nombre de trésors dans *K*. S'il n'y a plus de chasses au trésor à résoudre dans l'exécution en cours, la fonction fixe *N* et *K* à -1 ; dans ce cas, vous devez alors terminer votre programme avec le code de sortie 0. Notez que vous pouvez appeler cette fonction avant d'avoir trouvé tous les trésors de la chasse en cours, par exemple si vous essayez de ne résoudre que pour des points partiels.
- **enum** {*TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — ce sont les constantes utilisées dans les valeurs de retour de la fonction *Query* (voir ci-dessous).
- **int** *Query*(**int** *x*, **int** *y*) — si la case aux coordonnées (*x*, *y*) contient un trésor, cette fonction retourne *TREASURE*. Sinon, elle retourne la somme d'une ou plusieurs des constantes *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* et *DIR_DOWN*, indiquant quels déplacements la boussole retourne lorsqu'elle est posée sur la case (*x*, *y*). Les coordonnées *x* et *y* doivent être des entiers de 0 à $N - 1$. (Remarque : dans cette tâche, les coordonnées *y* augmentent du haut vers le bas.)

Après que *NextHunt* a retourné $N = K = -1$, votre programme ne doit plus appeler *NextHunt* ni *Query*. Il ne doit pas non plus appeler *Query* avant le premier appel à

NextHunt. Si votre programme ne respecte pas ces contraintes, ou s'il effectue plus de 1000 requêtes au sein d'une même chasse au trésor, ou s'il fournit des valeurs de x et/ou y hors de la plage autorisée lors de l'appel à *Query*, la bibliothèque terminera votre programme et retournera un run-time-error (RTE) pour le cas de test en cours.

Un trésor est considéré comme trouvé si vous avez interrogé au moins une fois la case qui le contient. Si votre programme appelle *NextHunt* avant d'avoir trouvé tous les trésors de la chasse en cours, cela n'est pas traité comme une erreur, mais cela affectera votre score (plus de détails sur le calcul du score ci-dessous).

Pour accéder à la bibliothèque, votre programme doit inclure le fichier d'en-tête `treasurehuntlib.h` :

```
#include "treasurehuntlib.h"
```

Vous pouvez télécharger ce fichier d'en-tête ici : `treasurehuntlib.h`.

Pour vous aider à développer votre solution, une implémentation simple de la bibliothèque est disponible ici : `treasurehuntlib-public.cpp`. Pour la compiler avec votre programme, ajoutez simplement son nom comme paramètre au compilateur, par exemple :

```
g++ foo.cpp treasurehuntlib-public.cpp
```

si `foo.cpp` est le nom du fichier contenant votre solution.

L'implémentation dans `treasurehuntlib-public.cpp` prend en charge, en plus des déclarations ci-dessus, une autre fonction appelée `void InitFromFile(const char *fileName)`, qui lit une liste de chasses au trésor depuis un fichier et vous permet de jouer avec celles-ci plutôt qu'avec ses propres chasses au trésor générées aléatoirement. Vous trouverez plus de détails à l'intérieur de `treasurehuntlib-public.cpp` lui-même.

Sur le serveur d'évaluation, une implémentation différente de la bibliothèque sera utilisée, vous ne devez donc faire aucune supposition sur le fonctionnement exact de l'implémentation. Vous pouvez toutefois supposer qu'elle ne pollue pas l'espace de noms global avec d'autres déclarations que celles listées (*NextHunt*, *Query* et les cinq constantes).

Votre code ne doit pas lire depuis l'entrée standard ni écrire sur la sortie standard, car celles-ci seront utilisées par notre implémentation de la bibliothèque sur le serveur d'évaluation pour communiquer avec le reste de l'environnement d'évaluation.

Entrée

Contraintes

- $2 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- Au sein d'une même exécution de votre programme, il y aura au plus 100 000 chasses au trésor.
- Au sein d'une même chasse au trésor, vous pouvez effectuer au plus 1000 requêtes.
- Le système de correction n'est pas adaptatif.

Sous-tâches

- Sous-tâche 1 (10 points) $K = 1$
- Sous-tâche 2 (30 points) $K = 2$
- Sous-tâche 3 (60 points) $K = 3$.

Calcul du score

Une sous-tâche peut être constituée de plusieurs cas de test (plusieurs exécutions de votre programme), et chaque cas de test peut être constitué de plusieurs chasses au trésor. Pour le calcul du score, toutes les chasses au trésor d'une sous-tâche donnée sont évaluées ensemble, indépendamment de la manière dont elles ont été initialement réparties entre les cas de test. Pour la i -ème chasse au trésor, notons la taille de la grille par $N_i \times N_i$, le nombre de trésors par K_i , le nombre de requêtes effectuées par votre programme par Q_i et le nombre de trésors trouvés par votre programme par F_i . Notons de plus par S le nombre total de points disponibles pour cette sous-tâche. Alors le nombre de points attribués à votre programme pour cette sous-tâche est :

- Si votre programme a toujours trouvé tous les trésors (c'est-à-dire si $F_i = K_i$ pour tout i), son score dépend de $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ points, où } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Si votre programme n'a pas toujours trouvé tous les trésors (c'est-à-dire s'il existe un i tel que $F_i < K_i$), il reçoit

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ points.}$$

Autrement dit, vous obtenez la moitié des points pour avoir trouvé tous les trésors, et l'autre moitié pour les avoir trouvés en un minimum de requêtes. Pour un score parfait, votre solution devrait trouver tous les trésors en au plus $11 \lceil \log_2 N_i \rceil$ requêtes. Entre $11 \lceil \log_2 N_i \rceil$ et $20 \lceil \log_2 N_i \rceil$ requêtes, le score décroît linéairement ; et si votre solution effectue plus de $20 \lceil \log_2 N_i \rceil$ requêtes, elle n'obtiendra que la première moitié des points, pour avoir trouvé tous les trésors. (Les symboles $\lceil \cdot \rceil$ signifient que la valeur de $\log_2 N_i$ est arrondie à l'entier supérieur.)

Si les formules ci-dessus donnent un nombre non entier de points pour la sous-tâche, il sera arrondi à l'entier le plus proche.

Si votre programme a une erreur d'exécution ou ne respecte pas le protocole d'utilisation de la bibliothèque mentionné ci-dessus, il obtiendra 0 point pour toute la sous-tâche. Pour recevoir des points partiels pour la découverte d'un sous-ensemble des trésors, votre programme doit donc terminer la recherche proprement en appelant *NextHunt*.

Exemple

Appel	Valeur de retour
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$